

LAB ACTIVITY (Exp-2&3)

1. Create a linked list by taking input from the user.
2. Find the middle of the linked list without using the length function.
3. Perform the deletion operation at the beginning of the linked list

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} node;

node* head = NULL;

void createnode() {
    node *q, *temp;
    temp = (node*)malloc(sizeof(node));
    printf("Enter data:");
    scanf("%d", &temp->data);
    temp->next = NULL;

    if (head == NULL) {
        head = temp;
    } else {
        q = head;
        while (q->next != NULL) {
            q = q->next;
        }
        q->next = temp;
    }
}

void display(node* h) {
    int k=1;
    while (h != NULL) {
        printf("Data %d: %d\n", k, h->data);
        h = h->next;
        k++;
    }
}

void displayMiddle(node *head){
    node *slow,*fast;
    slow=fast=head;
    if(head==NULL){
        printf("The list is empty");
    }
    while(fast!=NULL && fast->next!=NULL){
        fast=fast->next->next;
        slow=slow->next;
    }
    printf("\nMiddle data:%d",slow->data);
}
```

```

}

node* deleteatBegin(node *head){
    if (head == NULL) {
        printf("\nList is already empty!\n");
        return NULL;
    }
    node *temp = head;
    head = head->next;
    free(temp);
    return head;
}

int main() {
    int n, i;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        createnode();
    }

    printf("Linked list contents:\n");
    display(head);
    displayMiddle(head);
    head = deleteatBegin(head);
    printf("\nNew list after deletion at the beginning:\n");
    display(head);

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab2(1308)> cd "d:\Data structures and algorithm\lab3(0309)" ; if ($?) { gcc createlinked.c -o createlinked }
Enter the number of nodes: 5
Enter data:10
Enter data:20
Enter data:30
Enter data:40
Enter data:50
Linked list contents:
Data 1: 10
Data 2: 20
Data 3: 30
Data 4: 40
Data 5: 50

Middle data:30
New list after deletion at the beginning:
Data 1: 20
Data 2: 30
Data 3: 40
Data 4: 50
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab3(0309)>

```

4. Reverse a linked list

- Entire linked list
- Reversing the linked list using given positions of start and stop

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} node;

node* head = NULL;

void createnode() {
    node *q, *temp;
    temp = (node*)malloc(sizeof(node));
    printf("Enter data:");
    scanf("%d", &temp->data);
    temp->next = NULL;

    if (head == NULL) {
        head = temp;
    } else {
        q = head;
        while (q->next != NULL) {
            q = q->next;
        }
        q->next = temp;
    }
}

void display(node* h) {
    int k=1;
    while (h != NULL) {
        printf("Data %d: %d\n", k, h->data);
        h = h->next;
        k++;
    }
}

node * reverseLinkedList(node *head){
    node *pnode, *cnode, *nnode;
    pnode=nnode=NULL;
    cnode=head;
    while(cnode)
    {
        nnode=cnode->next;
        cnode->next=pnode;
        pnode=cnode;
        cnode=nnode;
    }
    return pnode;
}
```

```

}

node* reverseLinkedListPos(node* head, int m, int n) {
    node *pnode = NULL, *cnode = head, *nnode = NULL;
    node *tempnode1 = NULL, *tempnode2 = NULL;
    int k = 1;

    if (m == 1) {
        tempnode1 = NULL;
        tempnode2 = cnode;
    } else {
        while (k < m) {
            tempnode1 = cnode;
            cnode = cnode->next;
            k++;
        }
        tempnode2 = cnode;
    }

    pnode = NULL;
    while (cnode && m <= n) {
        nnode = cnode->next;
        cnode->next = pnode;
        pnode = cnode;
        cnode = nnode;
        m++;
    }

    if (tempnode1)
        tempnode1->next = pnode;
    else
        head = pnode;

    tempnode2->next = cnode;

    return head;
}

int main() {
    int n, i;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        createnode();
    }

    printf("Linked list contents:\n");
    display(head);

    head=reverseLinkedList(head);
    printf("Reversed Linked List:\n");
    display(head);

    int m, o;
    printf("Enter the positions you want to reverse the Linked List:");
    scanf("%d %d", &m, &o);
    head=reverseLinkedListPos(head, m, o);
}

```

```
printf("Reversed Linked List:\n");  
display(head);  
  
return 0;  
}
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab3(0309)> cd "d:\S  
tructures and algorithm\lab4(1009)\\" ; if ($?) { gcc reverselinkedlist.c -o  
($?) { .\reverselinkedlist }  
Enter the number of nodes: 5  
Enter data:10  
Enter data:20  
Enter data:30  
Enter data:40  
Enter data:50  
Linked list contents:  
Data 1: 10  
Data 2: 20  
Data 3: 30  
Data 4: 40  
Data 5: 50  
Reversed Linked List:  
Data 1: 50  
Data 2: 40  
Data 3: 30  
Data 4: 20  
Data 5: 10  
Enter the positions you want to reverse the Linked List:2  
4  
Reversed Linked List:  
Data 1: 50  
Data 2: 20  
Data 3: 30  
Data 4: 40  
Data 5: 10  
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab4(1009)> |
```